実験記録タブ UI設計書

メタデータ

- **画面ID**: ExperimentRecordTab
- **画面名**: 実験記録タブ
- **ファイルパス**: `frontend/src/components/ExperimentWorker/ExperimentRecordTab.tsx`
- **利用権限**: 実験作業者（experiment_worker）
- **作成日**: 2026-01-19
- **最終更新日**: 2026-03-11

---

1. 概要

1.1 画面の目的

実験作業者が実験データを記録するためのメインタブ画面です。プロジェクト、メソッド、タスクを選択し、トランジション単位で実験データを記録します。

1.2 主要機能

- プロジェクト・メソッド・タスクの選択
- トランジション一覧の表示（リスト形式/フロー図形式）
- 実験データの記録
- 問題報告

1.3 アクセス権限

- **対象ロール**: 実験作業者（experiment_worker）
- **必要条件**: ログイン済み、有効なセッション

---

2. 画面構成

2.1 画面イメージ



2.2 レイアウト構造

```
┌─────────────────────────────────────────┐
│ MethodTaskSelector（共通コンポーネント）      │
│ - プロジェクト選択（読み取り専用）             │
│ - メソッド選択                            │
│ - タスク選択                              │
│ - [選択] ボタン（モーダル表示）               │
├─────────────────────────────────────────┤
│ [表示切り替えボタン]                        │
│ - リスト表示 ⇄ フロー図表示                  │
├─────────────────────────────────────────┤
│ トランジション表示エリア（Card）              │
│ ┌─────────────────────────────────────┐ │
│ │ [リスト表示]                          │ │
│ │ - TransitionList コンポーネント       │ │
│ │                                     │ │
│ │ または                               │ │
│ │                                     │ │
│ │ [フロー図表示]                        │ │
│ │ - TransitionFlowDiagram コンポーネント │ │
│ └─────────────────────────────────────┘ │
└─────────────────────────────────────────┘

モーダル:
- MethodTaskSelectionModal（メソッド・タスク選択）
- DataRecordDialog（データ記録）
- IssueManagementModal（問題管理）
```

2.3 使用コンポーネント

Ant Design コンポーネント

- `Card` - トランジション表示エリアのコンテナ
- `Space` - 要素の間隔制御
- `Button` - 表示切り替えボタン
- `Typography.Title` - 見出し
- `Typography.Text` - テキスト表示
- `Empty` - 空状態表示
- `theme` - デザイントークン取得

カスタムコンポーネント

- `MethodTaskSelector` - メソッド・タスク選択UI
- `TransitionList` - トランジション一覧表示
- `TransitionFlowDiagram` - トランジションフロー図表示
- `DataRecordDialog` - データ記録モーダル

- `IssueManagementModal` - 問題管理モーダル
- `MethodTaskSelectionModal` - メソッド・タスク選択モーダル

アイコン

- `ExperimentOutlined` - 実験アイコン（空状態）
- `CheckSquareOutlined` - チェックボックスアイコン（空状態）
- `PartitionOutlined` - フロー図アイコン
- `UnorderedListOutlined` - リストアイコン
- `BarChartOutlined` - グラフアイコン

---

3. コントロール一覧

3.1 入力項目

項目名	タイプ	必須	初期値	説明
selectedProject	表示のみ	○	props経由	選択中のプロジェクトID（親から受け取り）
selectedMethodId	Select	○	null	選択中のメソッドID
selectedTaskId	Select	○	''	選択中のタスクID
viewMode	Radio	-	'list'	表示モード（'list' or 'flow'）

3.2 ボタン

ボタン名	タイプ	表示条件	アクション
選択	default	プロジェクト選択時	メソッド・タスク選択モーダルを開く
フロー図に切り替え	default + PartitionOutlined	リスト表示時 & メソッド・タスク選択済み	フロー図表示に切り替え
一覧表示に切り替え	default + UnorderedListOutlined	フロー図表示時 & メソッド・タスク選択済み	リスト表示に切り替え

3.3 表示項目

空状態メッセージ

状態	アイコン	タイトル	メッセージ
プロジェクト未選択	ExperimentOutlined (48px)	🧪 実験記録	プロジェクトを選択してください
メソッド未選択	ExperimentOutlined (48px)	📋 トランジション一覧	メソッドを選択してください
タスク未選択	CheckSquareOutlined (48px)	📋 トランジション一覧	タスクを選択してください

トランジション表示

- **リスト表示**: `TransitionList` コンポーネントで一覧表示
- **フロー図表示**: `TransitionFlowDiagram` コンポーネントでフロー図表示

---

4. イベント

4.1 ボタンクリックイベント

イベント名	トリガー	処理内容
handleRecordClick	トランジションの記録ボタンクリック	選択したトランジションを設定し、DataRecordDialogを開く
handleIssueClick	トランジションの問題報告ボタンクリック	選択したトランジションを設定し、問題一覧を取得してIssueManagementModalを開く
handleSwitchToFlow	フロー図切り替えボタンクリック	viewModeを'flow'に変更
handleSwitchToList	一覧表示切り替えボタンクリック	viewModeを'list'に変更

4.2 モーダルイベント

イベント名	トリガー	処理内容
handleDataRecordModalClose	データ記録モーダルを閉じる	モーダルを非表示、選択トランジションをクリア
handleIssueModalClose	問題管理モーダルを閉じる	モーダルを非表示、選択トランジションをクリア
handleDataSave	データ保存完了コールバック（DataRecordDialog の onSave から呼ばれる）	モーダルを閉じ、`fetchProgress()` で進捗データを再取得してトランジション一覧を最新状態に更新
handleSubmitIssue	問題報告送信	`createIssue()` APIを呼び出して問題を登録。成功後に問題一覧を再取得
handleSubmitComment	コメント送信	`addComment()` APIを呼び出してコメントを投稿。成功後に問題一覧を再取得

4.3 選択イベント

イベント名	トリガー	処理内容
onSelectMethod	メソッド選択変更	親コンポーネントのコールバック呼び出し
onSelectTask	タスク選択変更	親コンポーネントのコールバック呼び出し

---

5. データフロー

5.1 Props（親から受け取るデータ）

```
interface ExperimentRecordTabProps {
  selectedProject: string;                    // 選択中のプロジェクトID
  selectedProjectName: string;                // 選択中のプロジェクト名（API呼び出しに使用）
  onProjectChange: (value: string) => void;   // プロジェクト変更ハンドラ
  selectedMethodId: string | null;            // 選択中のメソッドID
  onSelectMethod: (methodId: string | null) => void; // メソッド選択ハンドラ
  selectedTaskId: string;                     // 選択中のタスクID
  onSelectTask: (taskId: string) => void;     // タスク選択ハンドラ
  onTaskCreated: (newTaskId: string) => void; // タスク作成ハンドラ
  selectedMethod: Method | null;              // 選択中のメソッドオブジェクト
  /** 選択中プロジェクトのステータス（数値）: 0=計画中, 1=タスク実行待ち, 2=タスク実行中, 3=承認待ち, 4=完了, 5=異常終了 */
  selectedProjectStatus: number | null;
  /** みなし進捗使用フラグ（organization.use_deemed_progress）: false の場合はチェックボックスを非表示にしてステータス更新を常時表示 */
  useDeemedProgress?: boolean;
}
```

5.2 State（内部状態）

```
// モーダル表示状態
const [modalOpen, setModalOpen] = useState(false);

// 表示モード
const [viewMode, setViewMode] = useState<ViewMode>('list');

// データ記録モーダル状態
const [dataRecordModalOpen, setDataRecordModalOpen] = useState(false);
const [selectedTransition, setSelectedTransition] = useState<Transition | null>(null);

// 問題管理モーダル状態
const [issueModalOpen, setIssueModalOpen] = useState(false);
const [issues, setIssues] = useState<Issue[]>([]);
const [issueLoading, setIssueLoading] = useState<boolean>(false); // 問題一覧取得中のローディング状態

// API連携の状態管理
const [loading, setLoading] = useState<boolean>(false);           // タスク一覧取得中のローディング状態
const [methods, setMethods] = useState<Method[]>([]);             // APIから取得したメソッド一覧
const [tasks, setTasks] = useState<Task[]>([]);                   // APIから取得したタスク一覧
const [projectId, setProjectId] = useState<string | null>(null);  // APIから取得したDBのプロジェクトID
const [progressData, setProgressData] = useState<Record<string, unknown>[]>([]); // 進捗APIから取得したトランジション進捗データ
```

5.3 算出データ

```
/**
 * selectedProjectStatus（数値）を ProjectStatus 型文字列にマッピング
 * 0=計画中 → null, 1=タスク実行待ち → 'waiting_task',
 * 2=タスク実行中 → 'executing_task', 3=承認待ち → 'waiting_approval',
```

```
 * 4=完了 → 'completed', 5=異常終了 → null
 */
const mapNumericStatusToProjectStatus = (status: number | null): ProjectStatus | null => { ... };

// prop から受け取った数値ステータスを ProjectStatus 型に変換
const projectStatus: ProjectStatus | null = mapNumericStatusToProjectStatus(selectedProjectStatus);

// 記録・参照可能状態判定
// isRecordViewableStatus() は ['waiting_task', 'executing_task', 'waiting_approval'] を活性とする
// projectStatus が null（取得前/エラー）の場合は安全側に倒して true（活性）とする
const isProjectRecordable = projectStatus === null || isRecordViewableStatus(projectStatus);

// selectedMethodId でフィルタしたタスク一覧（APIから取得した tasks State をフィルタ）
const filteredTasks = useMemo(() => {
  if (!selectedMethodId) return tasks;
  return tasks.filter(task => task.methodId === selectedMethodId);
}, [tasks, selectedMethodId]);

// 進捗データを含むトランジション一覧（progressData を起点に構築）
// selectedMethod.transitions は常に空配列のため、progressData を起点にマッピングする
// APIレスポンスフィールド対応:
//   material_count        → totalMaterialsCount  (MAIMLから取得した材料の定義数)
//   material_actual_count → inputMaterialsCount  (DBから取得した材料の実績数)
//   condition_count       → totalIngredientsCount (MAIMLから取得した条件の定義数)
//   condition_actual_count→ inputIngredientsCount (DBから取得した条件の実績数)
//   result_count          → totalResultsCount    (MAIMLから取得した結果の定義数)
//   result_actual_count   → inputResultsCount    (DBから取得した結果の実績数)
const transitionsWithProgress = useMemo(() => {
  if (!progressData || progressData.length === 0) return [];
  return progressData.map(progress => ({
    transitionId: progress.transition_id,
    transitionName: progress.transition_name ?? '',
    status: mapTransitionStatusToProgressStatus(progress.status),
    progressRate: progress.progress ?? 0,
    inputMaterialsCount: progress.material_actual_count ?? 0,
    totalMaterialsCount: progress.material_count ?? 0,
    inputIngredientsCount: progress.condition_actual_count ?? 0,
    totalIngredientsCount: progress.condition_count ?? 0,
    inputResultsCount: progress.result_actual_count ?? 0,
    totalResultsCount: progress.result_count ?? 0,
  }));
}, [progressData]);
```

5.4 使用するAPI

API	メソッド	エンドポイント	用途	実装状況
ワーカータスク一覧取得API	GET	`getWorkerTasks(projectName)` (projectApiService)	プロジェクト名でメソッド・タスク一覧を取得	✅ 完了
実験データ保存API	POST	（DataRecordDialog 内で実装）	実験データの一括保存	✅ 完了（子コンポーネントで実装）
ステータス更新API	PUT	（TransitionList 等の子コンポーネント内で実装）	トランジションステータスの更新	✅ 完了（子コンポーネントで実装）
問題報告API	POST	`/api/v1/issues`	問題報告の送信（`createIssue()`）	✅ 完了

コメント投稿API	POST	`/api/v1/issues/:issueId/comments`	問題へのコメント投稿（`addComment()`）	✅ 完了
問題一覧取得API	GET	`/api/v1/issues`	問題一覧取得（`getIssues()`）	✅ 完了
進捗データ取得API	GET	`/api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/transitions`	トランジションの進捗データ取得	✅ 完了

5.5 データソース

- `getWorkerTasks(projectName)` - `projectApiService` 経由でメソッド・タスク一覧を取得（実API）
- `getIssues({ projectId, methodId, taskNo, transitionId })` - `issueApiService` 経由でトランジションに紐づく問題一覧を取得（実API）
- `createIssue(...)` - `issueApiService` 経由で問題を新規登録（実API）
- `addComment({ projectName, issueId, comment })` - `issueApiService` 経由でコメントを投稿（実API）
- 進捗データ - `GET /api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/transitions` を `createApiClient()` で直接呼び出し（実API）

---

6. エラーチェック

6.1 バリデーション

項目	ルール	エラーメッセージ
プロジェクト選択	必須	「プロジェクトを選択してください」（空状態表示）
メソッド選択	必須	「メソッドを選択してください」（空状態表示）
タスク選択	必須	「タスクを選択してください」（空状態表示）

6.2 プロジェクトステータスによる制限

`isProjectRecordable` は `isRecordViewableStatus()` 関数（`types/experimentWorker.ts`）を使用して判定します。
`projectStatus` が `null`（取得前/エラー）の場合は安全側に倒して `true`（活性）とします。

プロジェクトステータス（数値）	ProjectStatus 文字列	記録・参照可否	説明
0（計画中）	null	○	null 扱い → 活性
1（タスク実行待ち）	waiting_task	○	記録・参照可能
2（タスク実行中）	executing_task	○	記録・参照可能
3（承認待ち）	waiting_approval	○	記録・参照可能

4（完了）	completed	×	完了済み - 記録不可
5（異常終了）	null	○	null 扱い → 活性

---

7. 画面遷移

7.1 遷移元

- ExperimentWorkerMenu（実験作業メニュー）から「実験記録」タブを選択

7.2 遷移先

なし（タブ内で完結）

7.3 モーダル遷移

```
ExperimentRecordTab
  ├→ MethodTaskSelectionModal（メソッド・タスク選択）
  │    └→ 選択後、メソッドIDとタスクIDを更新
  │
  ├→ DataRecordDialog（データ記録）
  │    └→ 保存後、モーダルを閉じる
  │
  └→ IssueManagementModal（問題管理）
       └→ 送信後、モーダルを閉じる
```

---

8. レスポンシブ対応

8.1 ブレークポイント

サイズ	幅	対応
Mobile	< 768px	縦スクロール、フロー図は横スクロール
Tablet	768px - 1024px	標準レイアウト
Desktop	> 1024px	標準レイアウト

8.2 モバイル対応

- フロー図表示時は横スクロール可能
- ボタンは縦配置（Space direction="vertical"）
- カードの余白調整

---

9. パフォーマンス考慮事項

9.1 最適化

- `transitionsWithProgress` は `useMemo` でメモ化
- 不要な再レンダリングを防ぐため、コールバック関数は適切にメモ化

9.2 データ量

- トランジション数: 最大100件程度を想定
- 進捗データ: 各トランジションごとに1件

---

10. 備考

10.1 特記事項

1. **表示モード切り替え**
   - リスト表示とフロー図表示を切り替え可能
   - 状態は `viewMode` で管理（'list' or 'flow'）
2. **進捗データのマージ**
   - 選択されたタスクに紐づく進捗データをトランジションにマージ
   - 進捗データがない場合はデフォルト値（未着手、進捗率0%）を設定
3. **プロジェクトステータスによる制御**
   - 承認待ち・タスク待ちステータスでは記録不可
   - `isProjectRecordable` フラグで制御

10.2 将来の拡張

1. バックエンドAPI統合（データ保存、問題報告）
2. リアルタイム進捗更新（WebSocket）
3. 一括データ記録機能
4. オフライン対応（データのローカル保存）
5. フロー図のカスタマイズ（ズーム、レイアウト調整）

10.3 関連ドキュメント

- TransitionList UI設計書
- TransitionFlowDiagram UI設計書
- DataRecordDialog UI設計書
- IssueManagementModal UI設計書
- MethodTaskSelector UI設計書

---

11. 変更履歴

バージョン	日付	変更内容	担当者
1.2.0	2026-03-11	ソースコードとの差分解消	Cline
		- Props定義に `selectedProjectName` ・ `selectedProjectStatus` ・ `useDeemedProgress` を追加	
		- State定義にAPI連携用State（ `loading` ・ `methods` ・ `tasks` ・ `projectId` ・ `progressData` ・ `issueLoading` ）を追加	
		- 算出データをモック参照から実API・数値ステータスマッピングベースに更新	
		- `isProjectRecordable` の判定ロジックを最新実装（ `isRecordViewableStatus()` ）に更新	
		- §6.2 プロジェクトステータス表をデータ数値対応表に更新	
		- API一覧を最新実装に合わせて更新（ `getWorkerTasks` 追加・進捗APIエンドポイント修正・TODO→完了済みに変更）	
		- データソースをモック参照から実APIサービスに更新	
		- `handleDataSave` ・ `handleSubmitIssue` ・ `handleSubmitComment` の実装内容をAPI呼び出しに更新	
1.1.0	2026-02-17	API仕様v2.1.0に準拠した修正	Cline
		- API一覧にエンドポイント列を追加	
		- 実験データ保存APIのエンドポイント更新	
		- ステータス更新APIのエンドポイント更新	
1.0.0	2026-01-19	初版作成	Cline